PROJECT REPORT

AI-assisted tool for evaluating aesthetic and composition quality of 3D scenes

Vladimir Vladinov

https://github.com/headedspy/ai-blender

TABLE OF CONTENTS

Introduction 3

1. Problem Definition 4
2. Proposed Solution 5
3. Personal Challenge 6
4. Final Implementation 7
5. Breakdown and findings 8
   - Data Generation 8
   - Training 10
   - Composition evaluation 12
   - Color scheme 13
   - Color feel 16
   - Reflection 18
6. Conclusion 20

INTRODUCTION

The project aims to utilize the power of artificial intelligence to assist artists in creating visually stunning and cohesive 3D scenes in Blender. The target group is both experienced artists looking for an extra set of eyes to review their work or a beginner trying to understand the principles of composition and color usage in 3D graphics. The AI can provide valuable insights and suggestions for improvement by analyzing the compositional elements and colors used in a Blender scene. The user interface is designed to be user-friendly and easy to use helping artists to craft more cohesive and aesthetically pleasing 3D worlds with greater efficiency.

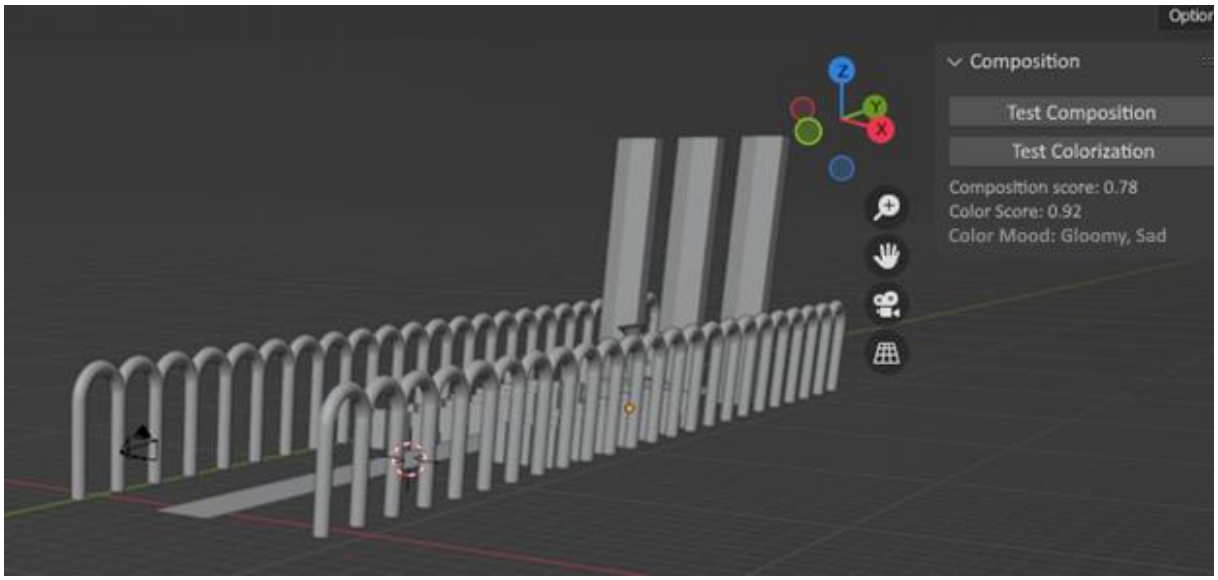
1. PROBLEM DEFINITION

Creating a 3D scene is an intricate process that requires a great deal of artistic skill and attention to detail. One of the major challenges that artists face when creating a 3D scene is figuring out the optimal composition, including the placement of objects and the use of negative space. This is especially important because the composition of a scene can greatly impact the overall aesthetic and cohesiveness of the work. In addition to composition, selecting an appropriate color palette that enhances the visual appeal and the emotional response of the scene is another significant factor to consider. However, finding the right combination of composition and color usage can be a daunting task, even for experienced artists. It requires a strong understanding of traditional art principles such as composition and color theory, as well as an eye for aesthetics and an ability to visualize the final product. For many artists, figuring out the optimal composition and appropriate color palette for a 3D scene can be a time-consuming and frustrating process, leading to wasted time and effort in the creative process.

The project aids with the composition and colorization steps by utilizing the power of AI. With it the user is able to evaluate the compositional integrity and color matching at any point of the scene creation process and look at the results immediately. Using the result, the artist would be able to iteratively alter the scene and re-evaluate to see if the alterations yielded positive or negative results.

2. PROPOSED SOLUTION

The tool is used as an add-on for blender. It should be able to evaluate the composition of a scene and its color palette separately. The scores should be presented in simple and intuitive way. The user should also be able to evaluate the scene at any given point and the same scene should yield the same results for the add-on to be useful. The project will contain two AI models: one designed for the composition rating, and one for the color palette. Additionally the colorization part should also report on the 'mood' of the colors used, that is the emotional response the artist aims to invoke via the scene.



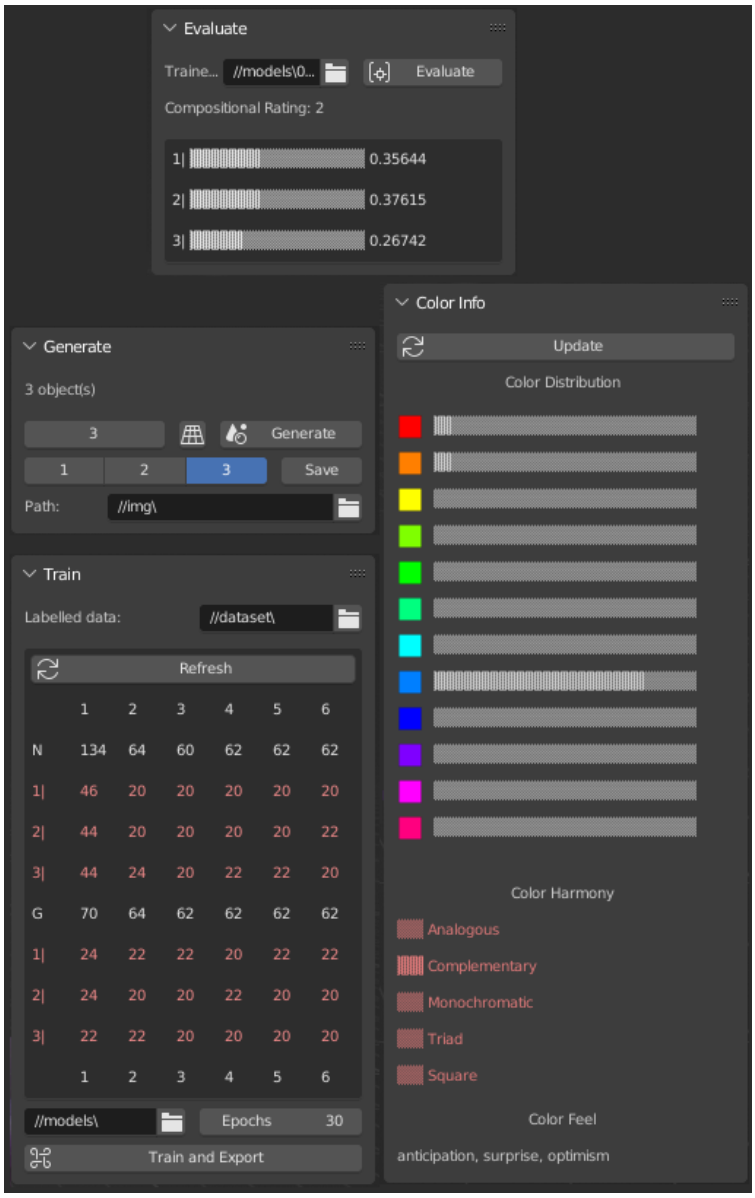
- **Test Composition:** This button will feed the image, generated by the virtual camera through the neural network and evaluate its composition quality.
- **Test Colorization:** This button will get only the colors from the image and send them through a second neural network to check for color balance.
- **Composition Score:** After evaluating the composition, the AI will return a number between 0 and 1. Higher number means better composition.
- **Color Score:** Upon checking the colorization the AI will return a number between 0 and 1. Higher number means better color balance and appropriately used hues.
- **Color Mood:** Additionally we can check how this specific choice of colors can be perceived by humans.

3. PERSONAL CHALLENGE

During this semester I set out a personal challenge to be more proficient in the world of AI and being able to confidently put machine learning in my tool belt. Additionally being a frequent user of Blender I wanted to create an add-on to help me in the future with creating 3d scenes. Every part of the add-on went through several iterations and in the end I chose which ones worked better for me personally, since the add-on will be used by me.

Coming from a software engineering background I was pretty confident in my technical and analytical skills, which allowed me to pick up Python and Tensorflow in a short amount of time. The initial weeks of the project I was busy with learning python, the theory behind neural networks and the Blender API. After that I started creating the first prototype and continued expanding on it.

4. FINAL IMPLEMENTATION



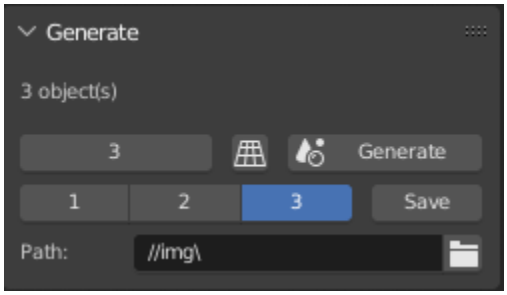
The final version of the add-on looks like this and has five different sub-menus. Each section is responsible for a certain task and can be closed off when not in use to save on vertical screen space.

One unique aspect of the project is that the user has the ability to generate their own dataset for training the AI to rate the composition of a 3D scene. This allows the user to incorporate their personal bias and aesthetic preferences into the AI's evaluation process. By providing a dataset of 3D scenes that they personally consider to be well-composed and visually appealing, the user can train the AI to recognize and prioritize similar elements in future 3D scenes. This allows for a greater degree of customization and personalization in the AI's evaluation process, enabling the user to fine-tune the AI's ratings to match their own artistic vision and standards. Overall, this feature allows the user to have more control over the AI's evaluation of 3D scenes and ensures that the AI's ratings align with the user's personal aesthetic preferences.

5. BREAKDOWN AND FINDINGS

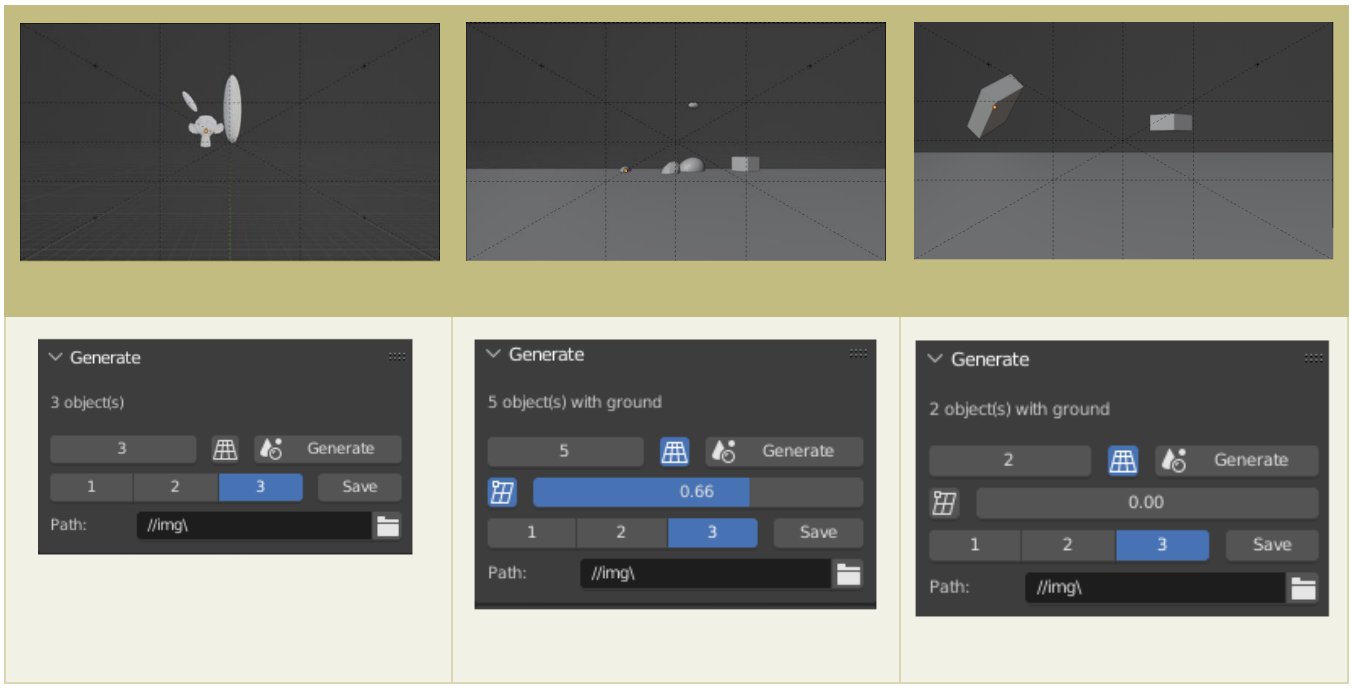
In the following sub-sections each sub-menu will be explained in details: its intended purpose, how to use it and how it works

Data Generation



The “Generate” sub-menu is used to generate a dataset for training the composition-evaluating AI on. With it the user can randomly generate scenes with a lot of variations and use it to incorporate their own bias into the AI rating. This is the first step for using the add-on. In order to start generating the dataset the user must first specify an empty folder where the images will be saved. After setting the folder you can specify the parameters of the scene. With the button you can change the amount of objects to be generated in the scene. You can generate up to 6 objects, after that the scene doesn’t really compositionally benefit or suffer from adding more objects and thus will only confuse the algorithm. The next option is the toggle button which turns on and off the ground generation. We want to train the algorithm on both images with a ground plane and aerial-type of composition that’s why we have this option. After toggling the ground generation on, a new option appears on the menu:

This is used to set the percentage of objects that should be placed on the ground. The rest of them will be generated in the air above the ground plane. The user can toggle the button on the left of the percentage bar, which will randomize the value on every scene. This will make it easier for the user to add another level of customization without having to manually set the value every time a new scene is generated. After customizing the scene the user can press the “Generate” button on the right, which will create a new randomized scene with the given parameters. Here are some examples of generated scenes and the given parameters:

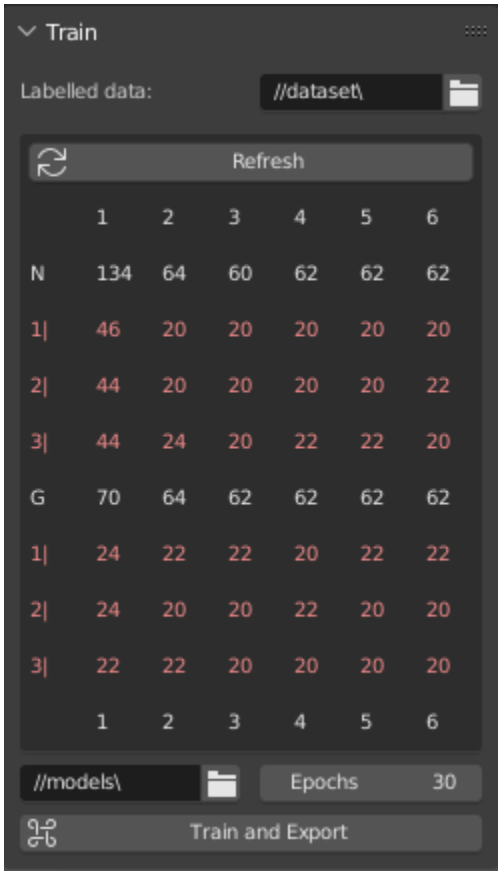


After the scene is generated, the user is prompted to rate in on a scale from 1 to three using the

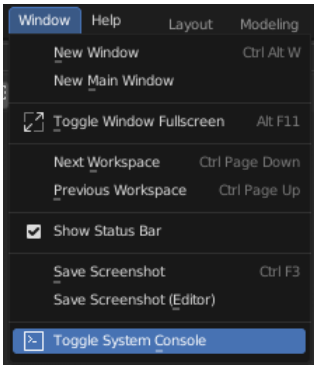
buttons below. Finally upon pressing the "Save" button an image in rendered and saved in the given output folder. The image's name consists of the rating and the parameters used to generate it in the following format: (Rating)_(Number of Objects)_(G if there's a ground plane and N if there's not).(uniquely generated UUID) For example:

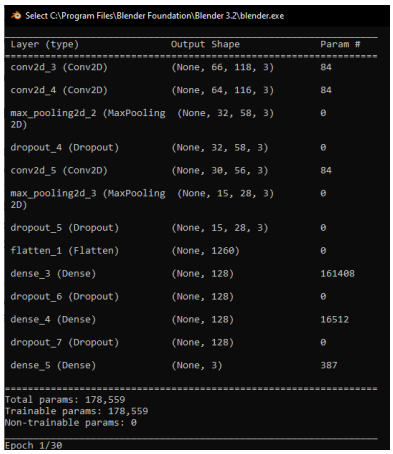
3_05_N.81b76f29-dc45-4565-8c6f-31c82b123080.png	Rating: 3, Number of objects: 5, Ground: No
3_01_G.82e7fa33-7082-48ad-8836-f70302ab5d03.png	Rating: 3, Number of objects: 1, Ground: Yes
1_03_G.d6e0479d-dd67-4b4a-a5ad-b71a49db5254.png	Rating: 1, Number of objects: 3, Ground: Yes

Training

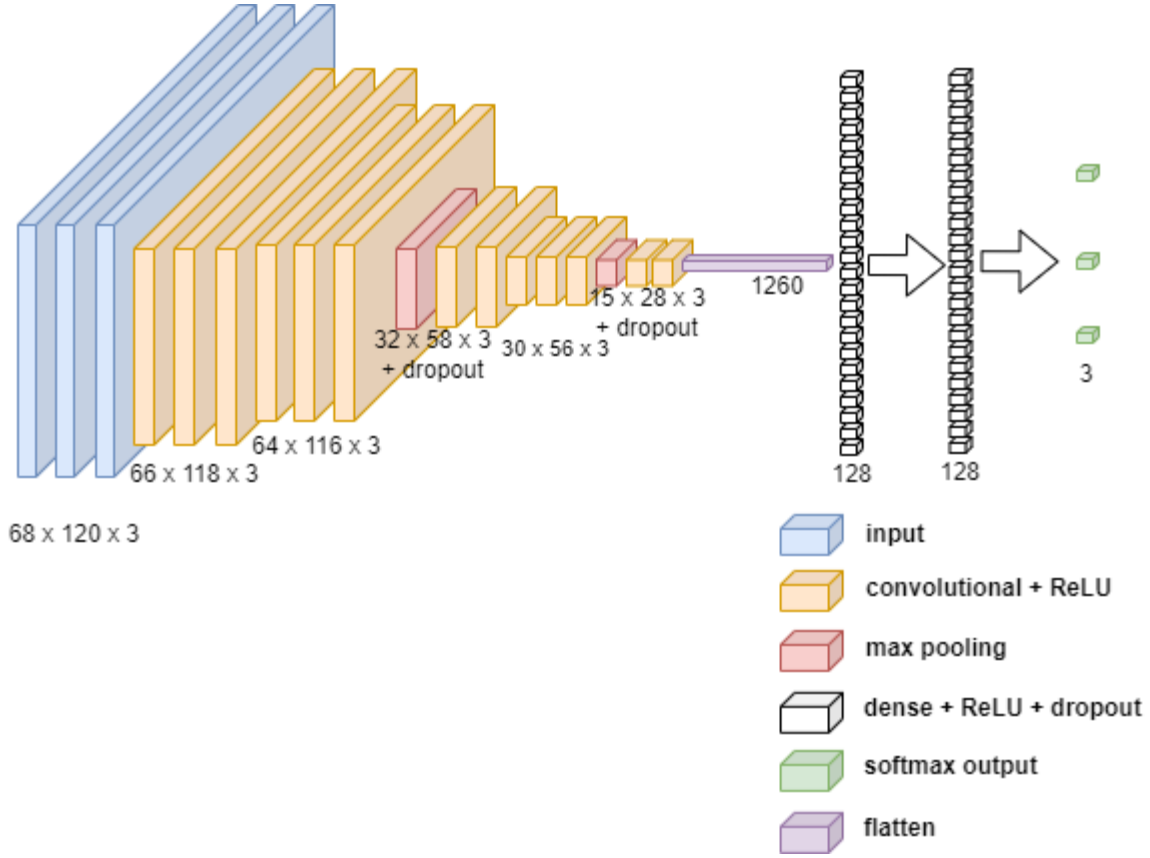


After a dataset has been generated it can be used to train a new model using the “Train” sub-menu. To use it, the user must first specify the folder, containing the dataset images. All the images in the folder should adhere to the above mentioned naming convention. After selecting the location and pressing the “Refresh” button, the matrix below gets filled with values. This table is used to see how many images of each combination were found in the dataset. For most optimal results it’s recommended to have at least 20 of each and keep all of them around the same number. In the table the 6 columns specify the number of objects and the rows – the ratings. The upper half is for the image without ground and the lower – for pictures containing the ground plane. Finally the white numbers show the sum of all images in the given “number of objects” category. For example in the given screenshots there are 134 images with 1 object without a ground plane, and from them 46 are rated with 1, 44 have rating 2, 44 are rated with 3. When the database gets selected the user must select export location for the trained model and the number of epochs to train it for. Finally in order to follow the progress of the training we can open the blender console and finally press the “Train and Export” button





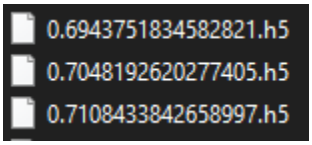
This will start the training process of the network. The model itself follows the given architecture:



The input is a 68 x 120 RGB image and the output is the given rating. During the training process we can track the accuracy and the loss in the console window:

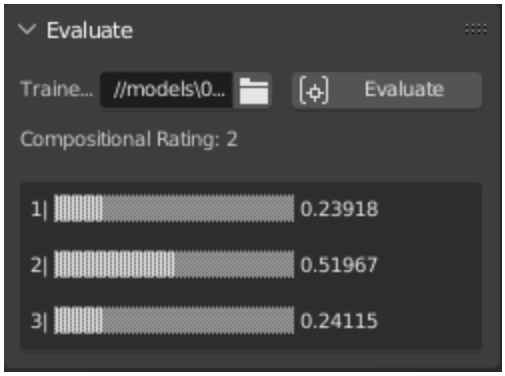
```
Epoch 1/30
21/21 [==============================] - 3s 79ms/step - loss: 1.4228 - accuracy: 0.3227 - val_loss: 1.3767 - val_accuracy: 0.3916
Epoch 2/30
21/21 [==============================] - 1s 62ms/step - loss: 1.3489 - accuracy: 0.3470 - val_loss: 1.3201 - val_accuracy: 0.3133
Epoch 3/30
21/21 [==============================] - 1s 63ms/step - loss: 1.2972 - accuracy: 0.4091 - val_loss: 1.2729 - val_accuracy: 0.4578
Epoch 4/30
21/21 [==============================] - 1s 62ms/step - loss: 1.2481 - accuracy: 0.4530 - val_loss: 1.2263 - val_accuracy: 0.4578
Epoch 5/30
21/21 [==============================] - 1s 63ms/step - loss: 1.1807 - accuracy: 0.4970 - val_loss: 1.1016 - val_accuracy: 0.6265
Epoch 6/30
21/21 [==============================] - 1s 62ms/step - loss: 1.0846 - accuracy: 0.5697 - val_loss: 1.0556 - val_accuracy: 0.6205
Epoch 7/30
21/21 [==============================] - 1s 62ms/step - loss: 0.9998 - accuracy: 0.6121 - val_loss: 0.9769 - val_accuracy: 0.5904
Epoch 8/30
21/21 [==============================] - 1s 62ms/step - loss: 0.9379 - accuracy: 0.6439 - val_loss: 0.9180 - val_accuracy: 0.6928
Epoch 9/30
21/21 [==============================] - 1s 62ms/step - loss: 0.9042 - accuracy: 0.6682 - val_loss: 0.9369 - val_accuracy: 0.6687
Epoch 10/30
21/21 [==============================] - 2s 78ms/step - loss: 0.8302 - accuracy: 0.6864 - val_loss: 0.8682 - val_accuracy: 0.6747
Epoch 11/30
21/21 [==============================] - 1s 62ms/step - loss: 0.7851 - accuracy: 0.7258 - val_loss: 0.8359 - val_accuracy: 0.6747
Epoch 12/30
21/21 [==============================] - 2s 79ms/step - loss: 0.7448 - accuracy: 0.7348 - val_loss: 0.8120 - val_accuracy: 0.6928
Epoch 13/30
21/21 [==============================] - 2s 76ms/step - loss: 0.7088 - accuracy: 0.7561 - val_loss: 0.8222 - val_accuracy: 0.6867
Epoch 14/30
21/21 [==============================] - 1s 62ms/step - loss: 0.6663 - accuracy: 0.7939 - val_loss: 0.8066 - val_accuracy: 0.6988
Epoch 15/30
21/21 [==============================] - 2s 89ms/step - loss: 0.6318 - accuracy: 0.7879 - val_loss: 0.8117 - val_accuracy: 0.6928
Epoch 16/30
21/21 [==============================] - 1s 64ms/step - loss: 0.6157 - accuracy: 0.8136 - val_loss: 0.8295 - val_accuracy: 0.6928
Epoch 17/30
21/21 [==============================] - 2s 72ms/step - loss: 0.5645 - accuracy: 0.8379 - val_loss: 0.8457 - val_accuracy: 0.6988
Epoch 18/30
21/21 [==============================] - 2s 82ms/step - loss: 0.5445 - accuracy: 0.8545 - val_loss: 0.8506 - val_accuracy: 0.7108
```

After the training is completed a new file with the extension .h5 will be created in the given directory. Its name will be the final evaluation of the model's accuracy:

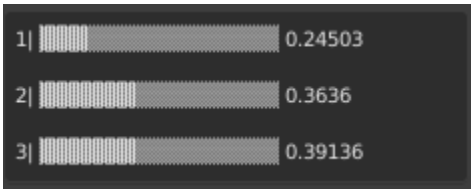


This allows the user to generate multiple models while experimenting with different datasets and epochs and finally select and use the one with the highest accuracy.

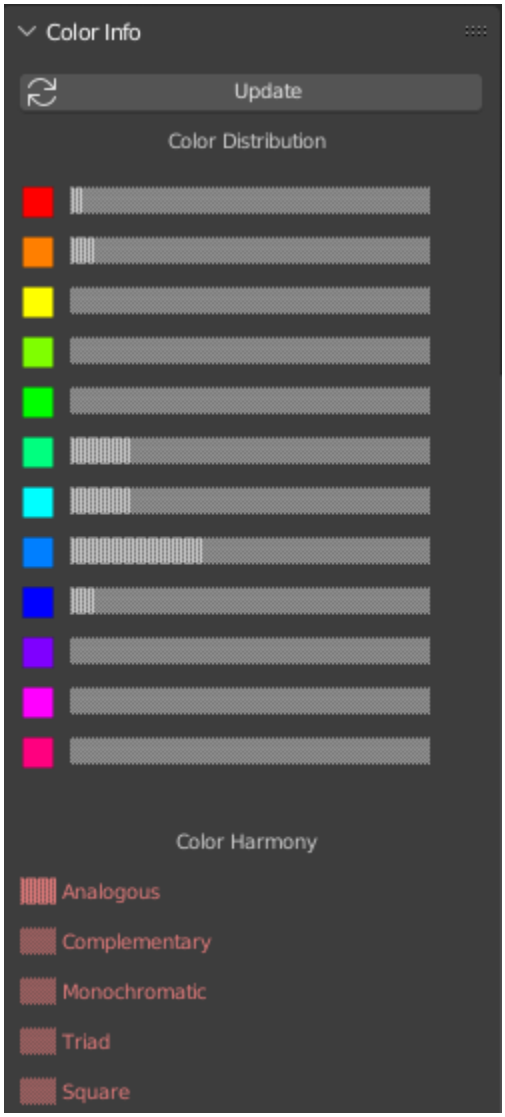
Composition evaluation



The trained model can be used via the "Evaluate" sub-menu. From it the user can select a .h5 file, representing the desired trained network and load it. Then by pressing the evaluate button an image will be created taken from the virtual camera and be fed into the neural network. The result and the confidence will be displayed below. In the given screenshot as an example the compositional aesthetic is rated as 2 with ~0.52 confidence. Twice as much as the other two options which are both at around 0.24. The ASCII progress bars were added for quick visual aid so the user can identify if the image falls between two categories easily. For example this result where the composition is rated between 2 and 3 but 3 is given as the final output:

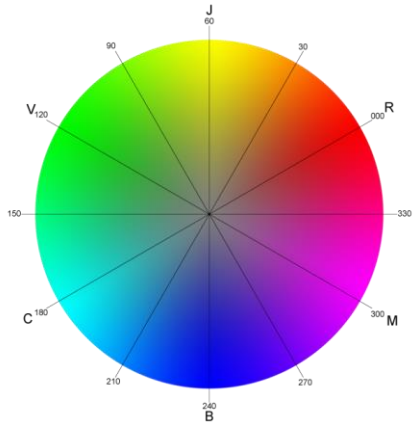


Color scheme

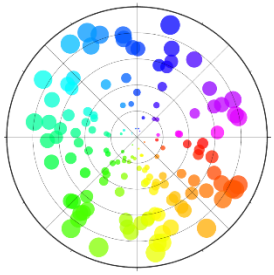


The color scheme can be found in the “Color Info” sub-menu. Upon pressing the “Update” button the user can receive information regarding the color palette used in the scene. Note that this is different from the compositional evaluation and while both are important for achieving aesthetically pleasing renders, they are kept as separate tools in the add-on. The color scheme retriever does not look at the arrangement, number or size of objects in the scene but only at the colors used.

The first information one can retrieve from the menu is the color distribution. With it the user can see which are the dominant colors and what’s the approximate ratio between the used hues. For example in the screenshot we can see that the main colors used are blue, followed by some cyan and greenish-blue. Also on the opposite side of the spectrum we see a really small amount of reds, perhaps on the highlights.

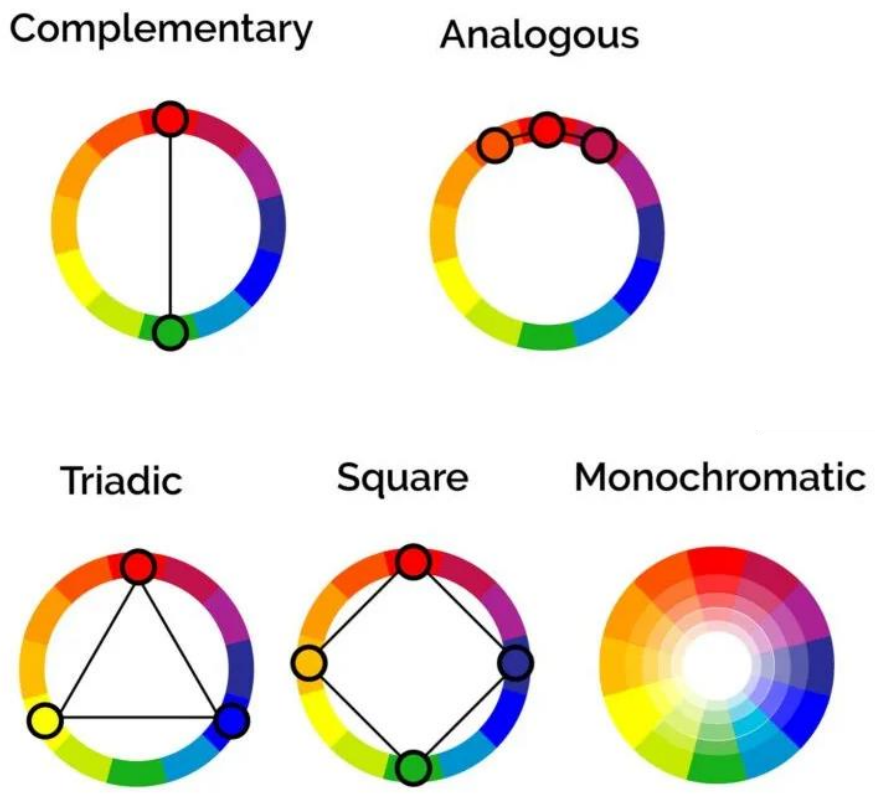


The colors chosen are taken from dividing the hue circle into 12. This gives us a certain degree for each color. Then we can iterate through every pixel on the image and create a polar scatter of each pixel’s hue while pushing it to the nearest of the 12 colors.

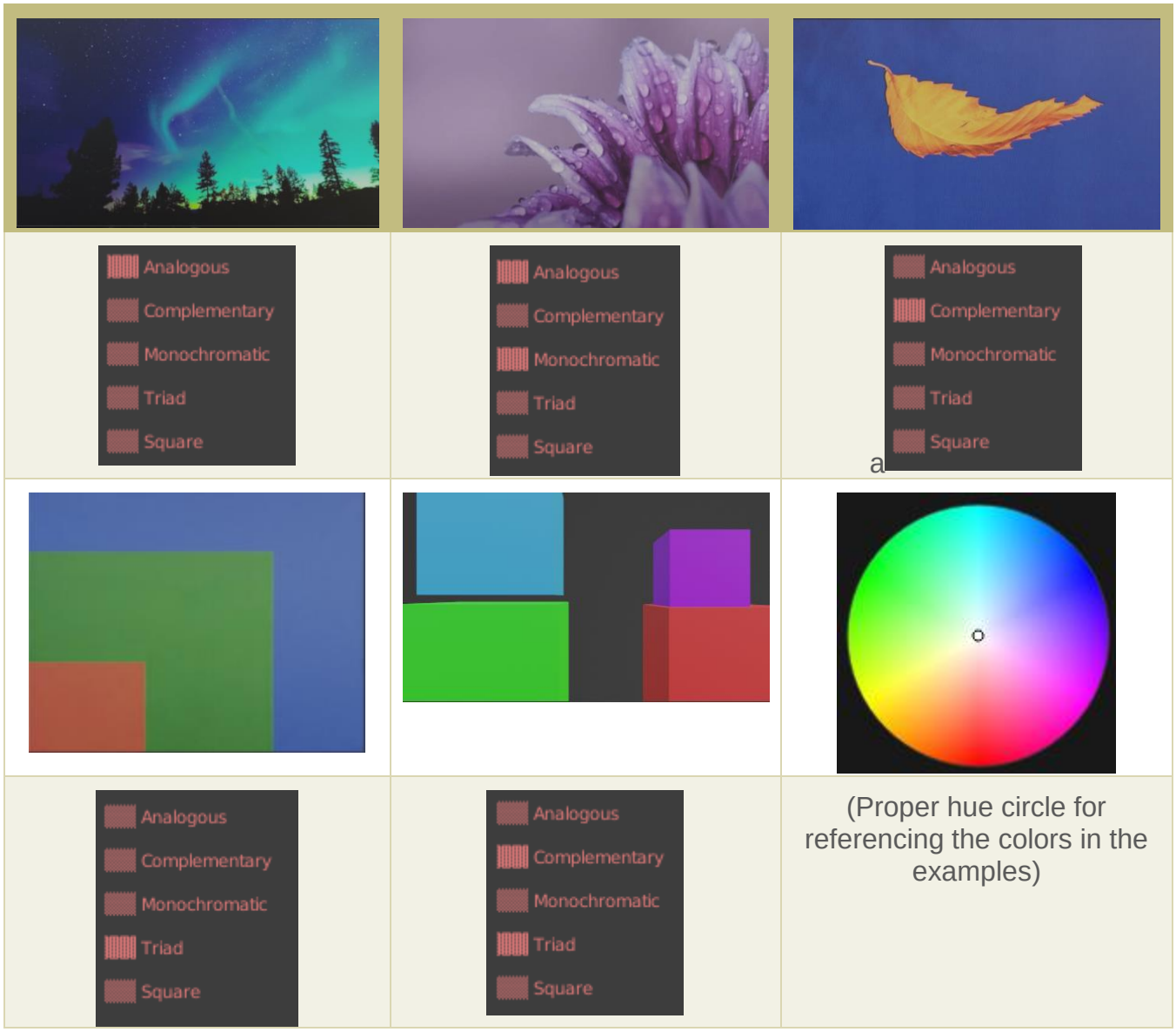


Finally by simply counting the number of pixels on each color and dividing it by the total number of pixels in the image we can visualize the ratio between hues in the image and display that information in human-readable format.

Additionally the add-on shows information about the existence of a color harmony in the primary hues of the image. The add-on is able to distinguish between the 5 basic harmonies:

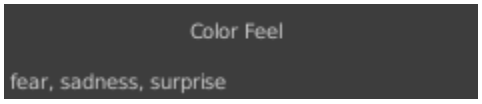


In order to do that, the add-on incorporates an implementation of ColorThief (https://github.com/fengsp/color-thief-py), a tool that utilizes a K-Means clustering algorithm to analyze the individual colors within an image and group them into a representative palette of the primary hues present. By distributing these colors on a color wheel, the system can then search for various combinations and alert the user whenever a visually pleasing and harmonious color arrangement is identified.



The first image contains hues mostly in the blue and greenish-blue area, that's why it's classified as analogous. In the second photo all colors come from the purple hue, thus it's categorized as monochromatic. Due to the multiple purple hues some of them can be closer to the blue or red tone, that's why the add-on finds analogous colors too. For the third image we have two colors on the opposite sides of the spectrum – orange and blue, defining a complementary color palette. The fourth painting yields a triad color palette. The fifth example contains 4 colors in a square formation, which can also be seen as two complementary palettes, that's why both square and complementary are found.

Color feel

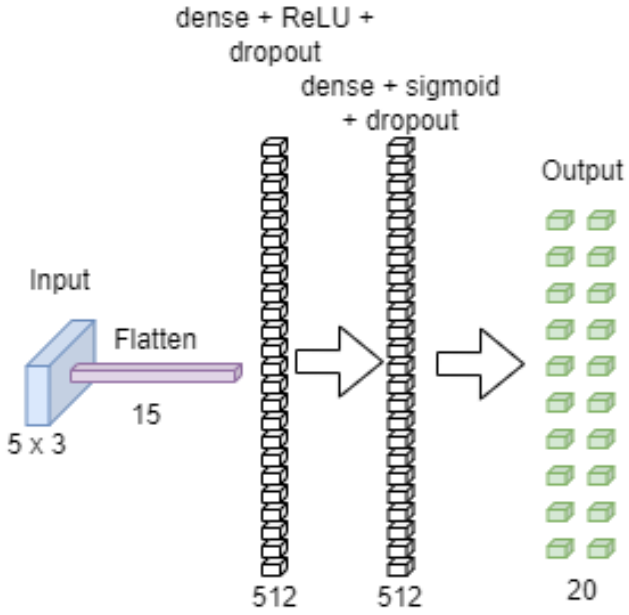


The final functionality of the add-on is to give the user a feel of what emotion the colors give to the viewer. It's located on the same "Color Info" sub-menu and is updated together with the color scheme. It uses a pre-trained model, which is located in the add-on folder. It works by using the same K-Means clustering algorithm but this time it returns the 5 most dominant colors in the image. Based on them a neural network returns three emotions, corresponding to the hues.

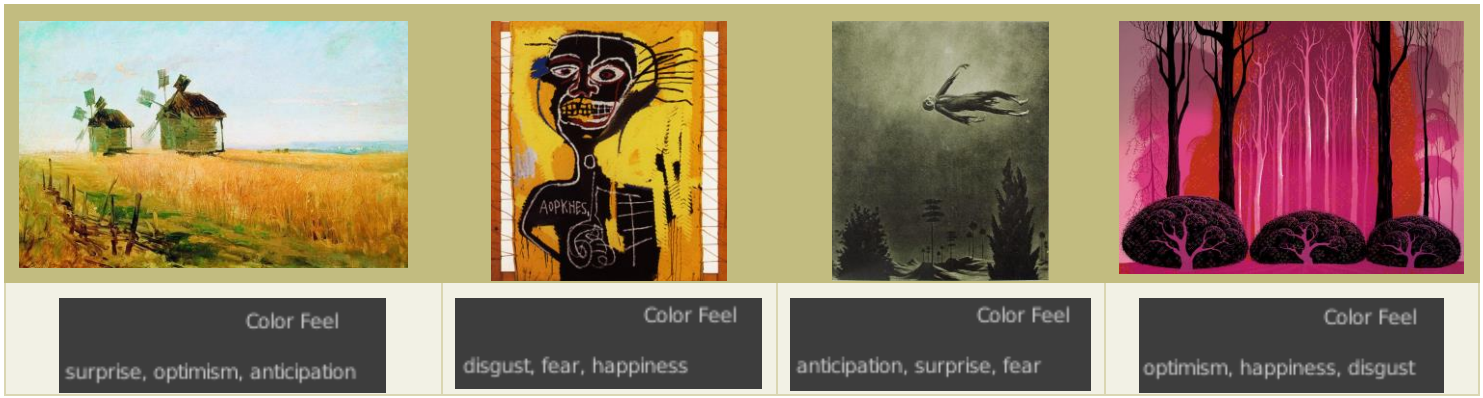
The algorithm was trained on the WikiArt Emotions Dataset (*http://saifmohammad.com/WebPages/wikiartemotions.html*)

This dataset contains over 4000 pieces of art, spanning over 22 categories (impressionism, realism, abstract expressionism etc.) and their corresponding annotations. They are labelled via crowdsourcing for one or more of twenty emotion categories and each piece of art has been annotated by at least 10 annotators.

Before using the dataset it had to be filtered. All pieces of art which were not a painting (sculptures, installations) were removed. Additionally if the painting contained a face we removed it, because we want to extract emotions based solely on the colors in the image, a face already contains emotions which is easily detectable by the viewer. Finally an average rating for every emotion for every painting was given based on the number of people that reported having said emotion when viewing it. This gave us a list of paintings and 20 numbers, corresponding to the average rating for every emotion. Finally we used this data to train a network with the following architecture, where the input is the 5 dominant colors of the image in RGB format and the output is the score for every emotion. The emotions themselves are the same as in the dataset: *agreeableness, anger, anticipation, arrogance, disagreeableness, disgust, fear, gratitude, happiness, humility, love, optimism, pessimism, regret, sadness, shame, shyness, surprise, trust, neutral*



After we use the model on a color palette it gives us a list of all emotions and a score from 1 to 0 for each of them. This list is sorted and the 3 highest-scoring emotions are presented to the user.



Reflection

Each sub-part of the add-on was created and tested individually using my previous knowledge of software engineering. I was also really comfortable with working on my own pace and my personal leadership qualities. It's my very first project using AI, Python and the Blender API but my previous experiences and my fast learning abilities helped me a lot.

During the development process a lot of initially planned things were changed. Here are the biggest alterations made to the add-on:

- The "Generate" section was initially able to generate with up to 10 objects and the score was from 1 to 5. This was changed since rating the composition of more than 6 objects becomes too fuzzy even for a human. The rating system also was quite uncertain around the middle ratings (2 and 4) and that's why it was reduced to only three numbers.
- The AI models went through several iterations with different optimizers, activation functions and loss functions until the current combination was found to yield the best results. Unfortunately there was no info on the internet for AI networks that have similar functionalities so it was mostly a process of trial and error.
- The Color Info panel initially included more harmonies such as "split-complimentary" and "rectangle" but due to them being just an extended version of "complimentary" and "square" respectively they were removed, which also made the UI and code a bit simpler.
- Initially the idea for the color harmonies was to build a third neural network to evaluate the colors, but since it's only color relations between degree on the hue circle it was a bit of an overkill and I decided to write a normal algorithm for it, checking the position of all colors' degrees and comparing them.

In the end I'm incredibly happy with the result. I managed to tackle every aspect of the machine leaning world: gathering data, filtering data, building a model, training and evaluating. The initially planned features are all present in the final product which shows I have picked an appropriate challenge for this

semester – not too easy but also not extremely difficult. I was able to tackle every learning outcome in some way except the societal impact, which is targeted more in the group project:

- **Investigative Problem Solving**: The initial problem was broken down into smaller pieces and each one of them was developed, tested and integrated seamlessly in the final solution. When no solution to a particular problem was found online I was able to create my own and incorporate it into the project.
- **Data Preparation**: I was able to include both dataset generation and data gathering procedures in the project. The latter of which had to undergo some filtering before it was deemed as useful.
- **Machine Teaching**: I was able to create two functional machine learning models without any prior knowledge of AI. They both yield useful results in their appropriate context and fit their initially intended purpose.
- **Data Visualization**: The dataset generation part can visualize to the user the number of unique images and give him important info if the model will be under-trained on some specific labels. Also the add-on gives some statistics such as the composition score and the color distribution in a simple user-friendly interface.
- **Reporting**: I tried to keep a basic information on the current progress of the project in the repository's README file and document every alteration accordingly. I could've been more active reporting to the teachers.
- **Personal Leadership**: I was able to use my programming background to properly plan the project scope from the beginning without exceeding my learning capacity. During the development process I worked in an iterative manner and documented the alterations made.
- **Personal Goal**: Without any prior knowledge of Python and AI as a whole I was able to deliver a somewhat-complex tool. I am now confident enough to put machine learning in my tool belt as a software engineer.

6. CONCLUSION

In the end we now have a reliable method for quickly evaluating the composition and color balance of an image while on the move. This will greatly assist with the artistic process, allowing me to concentrate more on other elements of the scene rather than worrying about the technical aspect of composition. Additionally, because the data used to train the AI model is customized to my personal artistic style, the AI will be biased towards my creative perspective. While this might be seen as a negative in some circumstances, in this particular context, it is actually a positive benefit as I plan to continue using this tool in the future. It will be particularly useful in helping me to make decisions about framing and positioning within the frame, as well as ensuring that the overall color scheme of the image is harmonious and aesthetically pleasing. By automating these tasks, I can save time and effort and focus on other important aspects of the scene, such as lighting, performance, and storytelling. I believe this tool will be a valuable asset in my work as a 3d creator, and I am excited to see how it can enhance my creative process in the future.